SISTEMA DE GESTÃO DE RETIRO

Documento de Arquitetura Técnica

Versão 3.1 (rascunho) — Março 2026

Stack: Node.js (Express) • PostgreSQL

`_meta`	Controle global: eventos, schemas de formulário, usuários admin, auditoria
`_sistema`	Infraestrutura transversal: sessões de jovens, documentos enviados
`retiro_AAAA`	Dados do evento: inscrições dinâmicas, equipes, enums, pagamentos

1. Visão Geral

Este documento descreve a arquitetura completa v3.0 do Sistema Web de Gestão de Retiro. A principal evolução em relação à versão anterior é a separação do banco de dados em três schemas PostgreSQL com responsabilidades distintas, eliminando o acoplamento entre tabelas de infraestrutura e tabelas específicas de cada evento.

1.1 Perfis de Usuário

Perfil	Autenticação	Descrição
Jovem	Não (cookie UUID)	Inscreve-se, paga, consulta equipe e envia documentos
Líder	Sim (login)	Gerencia jovens da sua congregação
Tesoureiro	Sim (login)	Visualiza e gerencia pagamentos
Administrador	Sim (login)	Acesso total, cria eventos e usuários

1.2 Princípios da Arquitetura v3.x

- Três schemas com responsabilidades exclusivas e bem definidas
- Tabelas fixas (sessões, documentos) vivem em _sistema, fora dos eventos
- Cada evento é um schema PostgreSQL isolado com apenas o que varia por evento
- O campo evento_slug em tabelas do _sistema é o único elo com o schema do evento
- Resolução de sessão do jovem é feita em _sistema e já retorna qual schema usar
- Nenhuma query precisa varrer múltiplos schemas para encontrar dados

2. Mapa dos Três Schemas

schema _meta — Controle Global

Tabela	Conteúdo
_meta.eventos	Todos os eventos cadastrados, com slug, status, padrão e datas
_meta.colunas_schema	Definição das colunas dinâmicas de cada evento (tipo, label, visibilidade)
_meta.usuarios_admin	Usuários administrativos (líder, tesoureiro, admin), função e congregação
_meta.audit_log	Registro de todas as ações administrativas
_meta.congregacoes	Registro de congregações

schema _sistema — Infraestrutura Transversal (fixa)

Tabela	Conteúdo
_sistema.sessoes_jovens	Tokens UUID de sessão do jovem, com evento_slug para resolução do schema
_sistema.documentos	Documentos enviados por jovens e líderes, com evento_id e codigo_unico

schema retiro_AAAA — Dados do Evento (dinâmico, um por evento)

Tabela	Conteúdo
retiro_AAAA.inscricoes	Inscrições com colunas dinâmicas definidas pelo schema do evento
retiro_AAAA.equipes	Equipes do evento com nome e cor
retiro_AAAA.enum_*	Tabelas de enumeradores customizados pelo administrador
retiro_AAAA.pagamentos	Pagamentos online e manuais vinculados às inscrições do evento

REGRA DE OURO	Uma tabela só entra no schema do evento se seu conteúdo muda entre eventos. Sessões e documentos existem independente de qual evento está ativo, por isso ficam em _sistema.

3. Schema _meta

3.1 Tabela: _meta.eventos

```
CREATE TABLE _meta.eventos (
  id                    SERIAL PRIMARY KEY,
  slug                  TEXT UNIQUE NOT NULL,   -- ex: retiro_2026
  nome                  TEXT NOT NULL,          -- ex: 'Retiro 2K20'
  ativo                 BOOLEAN DEFAULT TRUE,
  padrao                BOOLEAN DEFAULT FALSE,  -- apenas 1 TRUE por vez
  inscricoes_abertas    BOOLEAN DEFAULT FALSE,
  inscricoes_abertas_de TIMESTAMP,
  inscricoes_abertas_ate TIMESTAMP,
  criado_em             TIMESTAMP DEFAULT NOW(),
  criado_por            INTEGER REFERENCES _meta.usuarios_admin(id)
);
```

REGRA	Apenas um evento pode ter padrao = TRUE simultaneamente. O EventoService garante isso zerando todos os outros antes de marcar o novo.

3.2 Tabela: _meta.colunas_schema

```
CREATE TABLE _meta.colunas_schema (
  id           SERIAL PRIMARY KEY,
  evento_id    INTEGER NOT NULL REFERENCES _meta.eventos(id),
  nome         TEXT NOT NULL,      -- ex: tamanho_camiseta_enum_tamanhos
  label        TEXT NOT NULL,      -- ex: Tamanho da Camiseta
  tipo         TEXT NOT NULL,      -- text|number|date|boolean|email|enum
  obrigatorio  BOOLEAN DEFAULT TRUE,
  visibilidade TEXT DEFAULT 'public',  -- public | restricted
  visivel_para TEXT[],                 -- ex: {lider,admin}
  ordem        INTEGER DEFAULT 0
);
```

3.3 Tabela: _meta.usuarios_admin

```
CREATE TABLE _meta.usuarios_admin (
  id             SERIAL PRIMARY KEY,
  nome           TEXT NOT NULL,
  login          TEXT NOT NULL UNIQUE,
  senha_hash     TEXT NOT NULL,
  funcao         TEXT NOT NULL CHECK (funcao IN ('lider','tesoureiro','admin')),
  congregacao_id INTEGER REFERENCES _meta.congregacoes(id), -- obrigatorio para funcao = lider
  ativo          BOOLEAN DEFAULT TRUE,
  senha_inicial  BOOLEAN DEFAULT TRUE,
  criado_em      TIMESTAMP DEFAULT NOW(),
  criado_por     INTEGER REFERENCES _meta.usuarios_admin(id),

  CHECK (
    funcao <> 'lider'
    OR congregacao IS NOT NULL
  )
);
```

REGRA	Não pode existir zero administradores ativos. O sistema bloqueia desativação ou rebaixamento de função se restar apenas 1 admin ativo.

3.4 Tabela: _meta.audit_log

```
CREATE TABLE _meta.audit_log (
  id           SERIAL PRIMARY KEY,
  evento_slug  TEXT,
  usuario_id   INTEGER REFERENCES _meta.usuarios_admin(id),
  acao         TEXT NOT NULL,  -- INSCRICAO_EDITADA, PAGAMENTO_MARCADO ...
  entidade     TEXT,           -- inscricoes, equipes, usuarios_admin ...
  entidade_id  TEXT,           -- UUID ou id numerico do registro
  payload      JSONB,          -- snapshot antes/depois da alteracao
  ip           TEXT,
  criado_em    TIMESTAMP DEFAULT NOW()
);
```

4. Schema _sistema

O schema _sistema armazena tabelas de infraestrutura que são transversais a todos os eventos. Seu conteúdo não depende do schema dinâmico de nenhum evento específico. O campo evento_slug presente em suas tabelas é o único vínculo com o schema do evento correspondente.

4.1 Tabela: _sistema.sessoes_jovens

```
CREATE TABLE _sistema.sessoes_jovens (
  id               SERIAL PRIMARY KEY,
  inscricao_codigo UUID NOT NULL REFERENCES retiro_AAAA.inscricoes(id),
  evento_slug      TEXT NOT NULL,  -- qual schema usar ao carregar a inscricao
  token            TEXT NOT NULL UNIQUE,
  expira_em        TIMESTAMP NOT NULL,
  criado_em        TIMESTAMP DEFAULT NOW()
);
```

Por que evento_slug aqui: quando o jovem retorna ao sistema com o cookie, o backend faz uma única query em _sistema.sessoes_jovens e já obtém o schema correto do evento. Não é necessário varrer todos os schemas de eventos para localizar o UUID. É utilizado evento_slug invés de evento_id pois estes dados são processados no cliente e não é uma boa prática de segurança disponibilizar o ID de uma entidade para o cliente.

```
-- Resolucao de sessao em uma unica query:
SELECT inscricao_codigo, evento_slug
FROM   _sistema.sessoes_jovens
WHERE  token = $1 AND expira_em > NOW();

-- Com evento_slug em maos, o backend define o search_path:
SET search_path TO retiro_2026;
SELECT * FROM inscricoes WHERE codigo_unico = $2;
```

NOTA	Ao buscar documentos de uma inscrição utilizando de search_path, o backend deverá processar primeiramente procurar se o search_path que será utilizado realmente existe no banco de dados.

4.2 Tabela: _sistema.documentos

```
CREATE TABLE _sistema.documentos (
  id               SERIAL PRIMARY KEY,
  inscricao_codigo UUID NOT NULL REFERENCES retiro_AAAA.inscricoes(codigo),
  evento_slug      TEXT NOT NULL,
  nome             TEXT NOT NULL,
  caminho          TEXT NOT NULL,  -- path no servidor ou URL no storage
  enviado_por      TEXT NOT NULL,  -- 'jovem' | 'lider'
  lider_id         INTEGER REFERENCES _meta.usuarios_admin(id),
  enviado_em       TIMESTAMP DEFAULT NOW()
);
```

NOTA	Ao buscar documentos de uma inscrição, o backend filtra por inscricao_codigo AND evento_slug. O evento_slug evita colisão no caso (improvável mas possivel) de dois UUIDs idênticos em eventos distintos.

5. Schema por Evento: retiro_AAAA

Cada evento criado pelo administrador gera automaticamente um schema PostgreSQL isolado. O nome do schema é o slug do evento (ex: retiro_2026). Dentro desse schema vivem apenas as tabelas cujo conteúdo é específico daquele evento.

5.1 Tabela: inscricoes

```
CREATE TABLE retiro_2026.inscricoes (
  id                              SERIAL PRIMARY KEY,
  codigo_unico                    UUID DEFAULT gen_random_uuid() NOT NULL UNIQUE,
  congregacao                     TEXT NOT NULL,
  equipe_id                       INTEGER REFERENCES retiro_2026.equipes(id),
  cancelado                       BOOLEAN DEFAULT FALSE,
  motivo_cancelamento             TEXT,
  criado_em                       TIMESTAMP DEFAULT NOW(),
  atualizado_em                   TIMESTAMP DEFAULT NOW(),
  -- Colunas dinamicas geradas a partir de _meta.colunas_schema:
  nome_completo                   TEXT,
  email                           TEXT,
  documento_pessoal               TEXT,
  tamanho_camiseta_enum_tamanhos INTEGER,  -- FK -> enum_tamanhos.id
  tipo_inscricao_enum_tipos       INTEGER   -- FK -> enum_tipos.id
);

CREATE INDEX idx_retiro_2026_inscricoes_congregacao
ON inscricoes (congregacao);
```

CONVENCA O ENUM	Coluna que referencia enum segue o padrão <descricao>_enum_<tabela>. O SchemaService extrai o nome da tabela tudo que vier apos _enum_ no nome da coluna.

5.2 Tabela: equipes

```
CREATE TABLE retiro_2026.equipes (
  id         SERIAL PRIMARY KEY,
  nome       TEXT NOT NULL,
  cor        TEXT NOT NULL,
);
```

5.3 Tabelas enum_*

```
CREATE TABLE retiro_2026.enum_tamanhos (
  id    SERIAL PRIMARY KEY,
  valor TEXT NOT NULL,
  meta  TEXT   -- informacao opcional (descricao, cor, codigo)
);

CREATE TABLE retiro_2026.enum_tipos (
  id    SERIAL PRIMARY KEY,
  valor TEXT NOT NULL,   -- ex: Diaria, Integral
  meta  TEXT
);
```

5.4 Tabela: pagamentos

```
CREATE TABLE retiro_2026.pagamentos (
  id                SERIAL PRIMARY KEY,
  inscricao_codigo UUID NOT NULL
                    REFERENCES retiro_2026.inscricoes(codigo_unico),
  tipo              TEXT NOT NULL,      -- online | manual
  modalidade        TEXT,               -- diaria | integral | outro
  gateway           TEXT,               -- ex: mercadopago
  gateway_id        TEXT,
  status            TEXT NOT NULL,      -- pending|approved|rejected|manual
  valor             DECIMAL(10,2),
  payload_raw       JSONB,              -- dados brutos do gateway
  marcado_manual    BOOLEAN DEFAULT FALSE,
  data_pagamento    DATE,
  forma_pagamento   TEXT,               -- pix|dinheiro|transferencia
  observacao        TEXT,
  marcado_por       INTEGER REFERENCES _meta.usuarios_admin(id),
  criado_em         TIMESTAMP DEFAULT NOW()
);
```

6. Pools de Conexão

Com três schemas, o backend mantém dois pools de conexão fixos e cria conexões de evento dinamicamente por requisição.

6.1 Pool _meta

```
// db/meta.js
const metaDb = new Pool({ ...pgConfig });
metaDb.on('connect', c =>
  c.query("SET search_path TO _meta")
);
module.exports = metaDb;
```

6.2 Pool _sistema

```
// db/sistema.js
const sistemaDb = new Pool({ ...pgConfig });
sistemaDb.on('connect', c =>
  c.query("SET search_path TO _sistema")
);
module.exports = sistemaDb;
```

6.3 Conexão dinâmica por evento

```
// db/evento.js
const pools = {};  // cache de pools por slug

function getEventoDb(slug) {
  if (!pools[slug]) {
    pools[slug] = new Pool({ ...pgConfig });
    pools[slug].on('connect', c =>
      c.query(`SET search_path TO ${slug}`)
    );
  }
  return pools[slug];
}
module.exports = { getEventoDb };
```

SEGURANÇA	O slug é validado contra _meta.eventos antes de ser usado em SET search_path. Nunca interpolar slug direto de input do usuário sem validação prévia.

7. Roteamento Multi-Evento

A URL de inscrição pode ser acessada de duas formas. Em ambos os casos o backend resolve o evento antes de carregar o schema.

7.1 Rota padrão — GET /inscricao

Consulta _meta.eventos WHERE padrao = TRUE AND ativo = TRUE AND inscricoes_abertas = TRUE. Se existir, carrega o schema e entrega o formulário. É o link divulgado para os jovens e nunca muda.

7.2 Rota por slug — GET /inscricao/:slug

Consulta _meta.eventos WHERE slug = :slug. Permite acessar qualquer evento diretamente. Útil para testes, vedado para administradores.

7.3 Lógica de resolução

```
// routes/public.js
router.get('/inscricao/:slug?', async (req, res) => {
  const evento = req.params.slug
    ? await EventoService.getBySlug(req.params.slug)
    : await EventoService.getEventoPadrao();

  if (!evento) return res.render('sem-evento');

  const schema = await SchemaService.getSchema(evento.id, evento.slug, 'public');
  res.render('formulario', { evento, schema });
});
```

8. Sessão do Jovem — Fluxo Completo

8.1 Criação da sessão após inscrição

Passo	Schema	Ação
1	retiro_AAAA	Valida e insere inscrição, gera codigo_unico (UUID)
2	_sistema	Insere token UUID em sessoes_jovens com evento_slug e expira_em
3	HTTP	Define cookie HTTP-only Secure SameSite=Strict com o token
4	Frontend	Redireciona para /minha-inscricao

8.2 Retorno do jovem com cookie

Passo	Schema	Ação
1	HTTP	Backend lê cookie com token UUID
2	_sistema	SELECT inscricao_codigo, evento_slug FROM sessoes_jovens WHERE token = $1
3	retiro_AAAA	SET search_path TO evento_slug; SELECT * FROM inscricoes WHERE codigo_unico = $2
4	_meta	Carrega colunas_schema do evento para saber quais campos exibir
5	retiro_AAAA	Resolve valores dos enums (busca em enum_*)
6	retiro_AAAA	Verifica pagamentos pelo codigo_unico
7	Frontend	Exibe inscrição com campos públicos, status de pagamento e equipe

8.3 Recuperação de acesso (cookie perdido)

Passo	Schema	Ação
1	Frontend	Jovem acessa /auth/recuperar e informa e-mail, CPF ou código e evento
2	retiro_AAAA	Backend localiza inscrição pelo dado informado
3	_sistema	Cria novo token em sessoes_jovens com o evento_slug correto
4	HTTP	Exibe código
5	HTTP	Define novo cookie; jovem acessa /minha-inscricao normalmente

9. SchemaService

O SchemaService é o componente central que interpreta as colunas armazenadas em _meta.colunas_schema e opera dinamicamente sobre o schema do evento.

9.1 getSchema

```
// services/SchemaService.js
async function getSchema(eventoId, slug, userRole = 'public') {
  const { rows } = await metaDb.query(
    `SELECT * FROM colunas_schema
     WHERE evento_id = $1 ORDER BY ordem`, [eventoId]
  );
  const db = getEventoDb(slug);
  const cols = await Promise.all(
    rows.filter(c => isVisible(c, userRole))
        .map(async c => {
          if (c.tipo === 'enum') {
            const tabela = 'enum_' + c.nome.split('_enum_')[1];
            const opts   = await db.query(`SELECT * FROM ${tabela}`);
            return { ...c, opcoes: opts.rows };
          }
          return c;
        })
  );
  return cols;
}
```

9.2 isVisible

```
function isVisible(col, userRole) {
  if (col.visibilidade === 'public') return true;
  if (col.visibilidade === 'restricted')
    return col.visible_to?.includes(userRole);
  return false;
}
```

10. Estrutura de Diretórios

```
/projeto
  /src
    /routes
      public.ts         <- GET /inscricao, /inscricao/:slug, /pagamento/webhook
      jovem.ts          <- /minha-inscricao, /documentos
      lider.ts          <- /admin/lider/*
      tesoureiro.ts     <- /admin/tesoureiro/*
      admin.ts          <- /admin/*
      auth.ts           <- /auth/login, /auth/logout, /auth/recuperar
    /controllers
    /services
      metaService.ts    <- CRUD em _meta
      sistemaService.ts <- sessoes_jovens e documentos em _sistema
      eventoService.ts  <- cria schema PostgreSQL, gerencia eventos
      schemaService.ts  <- le colunas, filtra por role, resolve enums
      paymentService.ts <- integração com gateway
      auditService.ts   <- grava em _meta.audit_log
      equipeService.ts  <- sorteio e gestao de equipes
    /middlewares
      authAdmin.ts      <- valida sessão administrativa
      authJovem.ts      <- valida cookie UUID (consulta _sistema)
      role.ts           <- verifica funcao do usuario
      congregacao.ts    <- restringe lider a propria congregacao
    /db
      /repos            <- repositórios para acesso rápido facilitado
      conn.ts
      meta.ts           <- Pool fixo, search_path = _meta
      sistema.ts        <- Pool fixo, search_path = _sistema
      evento.ts         <- Pool dinamico por slug
  app.ts
```

11. Rotas da API

11.1 Rotas Públicas (Jovem)

Método	Rota	Schemas envolvidos
GET	/inscricao	_meta
GET	/inscricao/:slug	_meta
POST	/inscricao/:slug	_meta, retiro_AAAA, _sistema
GET	/minha-inscricao	_sistema, retiro_AAAA, _meta
PUT	/minha-inscricao	_sistema, retiro_AAAA
GET	/minha-equipe	_sistema, retiro_AAAA
POST	/documentos	_sistema, _sistema
POST	/pagamento/iniciar	_sistema, retiro_AAAA
POST	/pagamento/webhook	retiro_AAAA
POST	/auth/recuperar	retiro_AAAA, _sistema

11.2 Rotas do Líder

Método	Rota	Descrição
GET	/admin/lider/inscricoes	Lista inscritos da sua congregação
POST	/admin/lider/inscricoes	Inscreve jovem manualmente
DELETE	/admin/lider/inscricoes/:id	Remove inscrição com motivo
POST	/admin/lider/documentos/:id	Adiciona documento a jovem
GET	/admin/lider/documentos/:id	Lista documentos de jovem

11.3 Rotas do Tesoureiro

Método	Rota	Descrição
GET	/admin/tesoureiro/pagamentos	Lista pagamentos com filtros
GET	/admin/tesoureiro/pendentes	Pendentes por congregação
PATCH	/admin/tesoureiro/pagamentos /:id/marcar	Marca como pago manualmente
GET	/admin/tesoureiro/gateway	Transações online

11.4 Rotas do Administrador

Método	Rota	Descrição
GET	/admin/dashboard	Visão consolidada
GET	/admin/inscricoes	Todas as inscrições com filtros dinâmicos
PATCH	/admin/inscricoes/:id	Edita inscrição
DELETE	/admin/inscricoes/:id	Cancela inscrição
GET	/admin/eventos	Lista todos os eventos
POST	/admin/eventos	Cria evento + schema PostgreSQL
PATCH	/admin/eventos/:id	Edita evento
PATCH	/admin/eventos/:id/padrao	Define como evento padrão
GET	/admin/eventos/:id/colunas	Lista colunas do schema
POST	/admin/eventos/:id/colunas	Adiciona coluna
DELETE	/admin/eventos/:id/colunas/:col_Id	Remove coluna
GET	/admin/equipes	Lista equipes
POST	/admin/equipes	Cria equipe
PATCH	/admin/equipes/:id	Edita equipe
DELETE	/admin/equipes/:id	Exclui equipe
POST	/admin/equipes/sortear	Distribui jovens automaticamente
PATCH	/admin/equipes/atribuir	Atribui jovem a equipe
GET	/admin/usuarios	Lista usuários administrativos
POST	/admin/usuarios	Cria usuário administrativos
PATCH	/admin/usuarios/:id	Edita usuário
PATCH	/admin/usuarios/:id/status	Ativa / desativa
GET	/admin/audit	Log de auditoria

12. Controle de Acesso

12.1 Matriz de Permissões

Funcionalidade	Jovem	Líder	Tesoureiro	Admin
Preencher formulário	✓	✓ (congr.)	–	✓
Editar própria inscrição	–	✓ (congr.)	–	✓
Ver própria inscrição	✓	–	–	✓
Ver inscrições da congr.	–	✓	–	✓
Ver todas as inscrições	–	–	✓*	✓
Editar qualquer inscrição	–	–	–	✓
Cancelar inscrição	–	✓ (congr.)	–	✓
Ver status de pagamento	✓ (própria)	✓ (congr.)	✓	✓
Realizar pagamento online	✓	✓	–	✓
Marcar pagamento manual	–	–	✓	✓
Ver equipe	✓	✓	–	✓
Gerenciar equipes	–	–	–	✓
Enviar documentos	✓	✓	–	✓
Gerenciar eventos e schema	–	–	–	✓
Criar usuários admin	–	–	–	✓
Ver audit log	–	–	–	✓

** Tesoureiro vê somente colunas com visibilidade pública ou marcadas como visíveis para tesoureiro.*

12.2 Middleware authJovem

```
// middlewares/authJovem.js
async function authJovem(req, res, next) {
  const token = req.cookies?.sessao_jovem;
  if (!token) return res.status(401).json({ error: 'Sem sessao' });

  // Consulta SEMPRE em _sistema, nunca no schema do evento
  const { rows } = await sistemaDb.query(
    `SELECT inscricao_codigo, evento_slug
     FROM sessoes_jovens
     WHERE token = $1 AND expira_em > NOW()`, [token]
  );
  if (!rows.length) return res.status(401).json({ error: 'Sessao expirada' });

  req.jovem = rows[0];   // { inscricao_codigo, evento_slug }
  next();
}
```

13. Segurança

Camada	Medida
Senhas	bcrypt (salt ≥ 12) ou Argon2 para usuários admin
Cookie jovem	HTTP-only, Secure, SameSite=Strict; token UUID v4 sem dados pessoais
SQL	Queries 100% parametrizadas; nenhuma concatenação de string
search_path	Slug validado contra _meta.eventos antes de usar em SET search_path
Webhook	Validação de assinatura HMAC do gateway antes de processar
CORS	Restrito ao domínio da aplicação
Rate limit	express-rate-limit nas rotas de login e inscrição
XSS / CSRF	Sanitização de inputs, CSP header, token CSRF em formulários
Auditoria	AuditService grava em _meta.audit_log toda ação administrativa
Senha inicial	Admin criado com senha_inicial = TRUE; obrigado a trocar no 1o login
Zero admins	Bloqueio de desativação se restar apenas 1 admin ativo
Privacidade	Cookie não contém dados pessoais; apenas token UUID opaco
Um cadastro	Bloqueia tentativa de inscrição com dados iguais ou semelhantes

14. Requisitos Não Funcionais

Categoria	Requisito
Disponibilidade	Sistema disponível 24/7 durante inscrições; backup automático diário
Desempenho	Respostas em até 3 segundos; suporte a múltiplos acessos simultâneos
Responsividade	Interface adaptável a mobile e desktop
Usabilidade	Feedback visual claro; navegação intuitiva; mensagens de erro descritivas
Privacidade	Dados sensíveis não expostos no browser; identificador sem dados pessoais
Integridade	Inscrição só confirmada após pagamento válido
Auditoria	Toda acao administrativa registrada com usuário, IP e snapshot de dados
Isolamento	Dados de cada evento completamente isolados em schema PostgreSQL próprio
Escalabilidade	Novos eventos criados sem impacto em dados de eventos anteriores

15. Próximos Passos

Ordem sugerida de implementação

- Criar os três schemas iniciais: _meta, _sistema e retiro_2026 de exemplo
- Implementar os três pools de conexão (meta.js, sistema.js, evento.js)
- Implementar EventoService: criação de schema PostgreSQL + tabelas por evento
- Implementar SchemaService: leitura de colunas, filtragem por role, resolução de enums
- Implementar SistemaService: CRUD de sessoes_jovens e documentos em _sistema
- Implementar rotas públicas: GET /inscricao, POST /inscricao/:slug
- Implementar middleware authJovem consultando _sistema.sessoes_jovens
- Implementar rotas do jovem: /minha-inscricao, /minha-equipe, /documentos
- Implementar autenticação administrativa e middleware de roles
- Implementar rotas do líder, tesoureiro e administrador
- Implementar integração com gateway de pagamento e webhook
- Implementar gestão de equipes e sorteio
- Implementar painel de gestão de eventos (criação, troca de padrão, clonagem)
- Testes de integração, auditoria e cenários de recuperação de acesso

RECOMENDAÇÃO FINAL	Comece criando manualmente os três schemas no PostgreSQL e inserindo um evento e algumas colunas diretamente via SQL. Valide o fluxo completo (jovem se inscreve, cookie criado em _sistema, jovem consulta inscrição) antes de construir qualquer interface administrativa.